Final Project Report

Greg Pologruto

2026-05-06

This study explores the development of a machine learning approach for detecting fraudulent online transactions with improved accuracy, efficiency, and scalability compared to traditional fraud detection methods. The dataset contains 7,500 synthetic digital payment transactions labeled as either fraudulent or legitimate, with 14 behavioral, financial, device, and risk-related predictor variables. Data preprocessing techniques, including cleaning and resampling, were applied to improve data quality and address class imbalance. Several machine learning models were implemented and compared, including Logistic Regression, K-Nearest Neighbors (KNN), Decision Trees, Random Forest, and Boosted Trees. Model performance was evaluated using metrics such as accuracy, balanced accuracy, F1-score, ROC curves, and sensitivity. Particular emphasis was placed on sensitivity because the positive class represents fraudulent transactions, making the true positive rate critical for effective fraud detection. The best-performing model was selected based on overall classification performance across these evaluation metrics.

Introduction

This study presents the development and evaluation of a machine-learning based approach to address the problem of detecting fraudulent online transactions. The objective of this research is to design a predictive classification model capable of improving accuracy, efficiency, and scalability compared to traditional methods.

The dataset used in this study consists of 7500 synthetic digital payment transactions labeled as either fraudulent or legitimate. Each row represents a single transaction and contains features about behavior, finance, device, and risk-related features. There are 14 features that can be used as predictor variables to predict if the transaction is fraudulent. Data preprocessing techniques such as cleaning and resampling were applied to enhance data quality. Several machine learning methods such as Logistic Regression, KNN, Decision Trees, Random Forest, and Boosted Trees were implemented and evaluated.

Model performance is assessed using standard evaluation metrics such accuracy, balanced accuracy, F-1 score, ROC curves, and most importantly sensitivity. Since in this case the positive class is fraudulent transactions, special focus will be put on true positive rate. The best model is selected using the highest performance metrics.

Methodology

Logistic Regression

Binary logistic regression is used when the response variable Y has only two possible outcomes. Instead of modeling Y directly, the model estimates the probability that an observation belongs to the positive class.

Let the predictor vector be $x = (x_1, x_2, \dots, x_p)$. The model estimates (EQ 1):

$$P(Y = 1 \mid X = x) = \frac{\exp(\beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p)}{1 + \exp(\beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p)}$$

This is known as the logistic (sigmoid) function, which ensures predicted probabilities lie between 0 and 1.

Log-Odds (Logit Formulation)

Rather than modeling probability directly, logistic regression models the log-odds (EQ 2):

$$\log\left(\frac{P(Y = 1 \mid X)}{1 - P(Y = 1 \mid X)}\right) = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p$$

Here: - $\frac{P(Y=1)}{1-P(Y=1)}$ is called the odds - The log of the odds is called the logit function

Parameter Estimation

The parameters β are estimated using maximum likelihood estimation (MLE). For n independent observations, the likelihood is (EQ 3):

$$L(\beta) = \prod_{i=1}^{n} p\,(x_i)^{y_i}\big(1 - p(x_i)\big)^{1-y_i}$$

where: - $y_i \in \{0,1\}$ - $p(x_i) = P(Y = 1 \mid X = x_i)$

Random Forest Classifier

Random forest is an ensemble learning method that builds on bagging by combining many decision trees to improve prediction accuracy and reduce variance. Instead of relying on a single tree, the model aggregates predictions from multiple trees trained on different subsets of the data.

Model Formulation

Given a training dataset, random forest constructs B decision trees using bootstrap samples. Let $\hat{f}_b(x)$ denote the prediction from the b-th tree. The final prediction is obtained by averaging (for regression) or majority voting (for classification) (EQ4):

$$\hat{f}_{RF}(x) = \frac{1}{B}\sum_{b=1}^{B} \hat{f}_b\,(x)$$

This averaging process reduces variance compared to a single decision tree.

Random Feature Selection

A key difference between bagging and random forests is that, at each split, only a random subset of predictors is considered. If there are p total predictors, a subset of size $m < p$ is selected at each split:

$$m \approx \sqrt{p}$$

This introduces randomness and reduces correlation between trees, leading to better overall performance.

Algorithm

1. Draw B bootstrap samples from the training data

2. For each bootstrap sample:
    - Grow a decision tree

- At each split, consider only m randomly selected predictors

3. Aggregate predictions across all trees:
   - Average (regression)
   - Majority vote (classification)

Out-of-Bag (OOB) Error

Each tree is trained on roughly $2/3$ of the data, leaving out the remaining observations. These unused observations can be used to estimate prediction error without cross-validation (EQ 5):

$$\text{OOB Error}_i = \frac{1}{|K_i|} \sum_{k \in K_i} L\left(\hat{f}_k(x_i), y_i\right)$$

where K_i is the set of trees that did not include observation i, and $L(\cdot)$ is a loss function.

Boosting

Boosting is an ensemble learning method that builds a strong model by sequentially combining many weak learners, typically decision trees. Unlike bagging and random forests, which reduce variance, boosting primarily aims to reduce bias by learning from previous errors.

Model Formulation

Boosting constructs a model as a weighted sum of B weak learners (trees). Let $\hat{f}^{(b)}(x)$ denote the prediction from the b-th tree. The final model is (EQ 6):

$$\hat{f}(x) = \sum_{b=1}^{B} \lambda \, \hat{f}^{(b)}(x)$$

where λ is the learning rate (shrinkage parameter) that controls how much each tree contributes to the model.

Algorithm

1. Initialize the model:

$$\hat{f}^{(0)}(x) = 0$$

2. For $b = 1, \ldots, B$:
   - Compute residuals: $r_i^{(b)} = y_i - \hat{f}^{(b-1)}(x_i)$

- Fit a decision tree $\hat{f}^{(b)}(x)$ to the residuals
- Update the model: $\hat{f}^{(b)}(x) = \hat{f}^{(b-1)}(x) + \lambda \hat{f}^{(b)}(x)$

3. Output the final model (EQ 7):

$$\hat{f}(x) = \sum_{b=1}^{B} \lambda \hat{f}^{(b)}(x)$$

This sequential learning process allows each new tree to focus on correcting the errors of the previous model.

Tuning Parameters

Boosting depends on several key parameters:

- B: number of trees
- d: depth of each tree (controls model complexity)
- λ: learning rate (shrinkage parameter)

Bayesian Additive Regression Trees (BART)

Bayesian Additive Regression Trees (BART) is an ensemble learning method that combines ideas from bagging and boosting within a Bayesian framework. Instead of building trees independently or sequentially, BART models the response as a sum of many small regression trees, each capturing a portion of the signal.

Model Formulation

BART represents the prediction function as a sum of K regression trees (EQ 8):

$$\hat{f}(x) = \sum_{k=1}^{K} \hat{f}_k(x)$$

Each individual tree is constrained to be small so that no single tree dominates the model. The overall prediction is obtained by aggregating contributions from all trees.

Iterative Bayesian Updating

At each iteration $b = 1, \ldots, B$, the model updates trees based on residuals from the current fit. Let $\hat{f}_k^{(b)}(x)$ denote the prediction from tree k at iteration b (EQ 9):

$$\hat{f}^{(b)}(x) = \sum_{k=1}^{K} \hat{f}_k^{(b)}(x)$$

Residuals are computed as (EQ 10):

$$r_i^{(b)} = y_i - \sum_{k' \neq k} \hat{f}_{k'}^{(b-1)}(x_i)$$

Each tree is updated using these partial residuals, allowing the model to gradually improve its fit.

Final Prediction

After a burn-in period of L iterations, predictions are averaged over the remaining iterations (EQ 11):

$$\hat{f}(x) = \frac{1}{B-L} \sum_{b=L+1}^{B} \hat{f}^{(b)}(x)$$

This averaging reduces variance and provides a stable estimate of the prediction function.

Tuning Parameters

BART depends on several key parameters:

- K: number of trees
- B: number of iterations
- L: number of burn-in iterations

---

k-Nearest Neighbors (KNN)

The k-nearest neighbors (KNN) classifier is a non-parametric method that assigns a class to a new observation based on the classes of its nearest neighbors in the predictor space.

For a new observation x_0, the algorithm identifies the set $N_k(x_0)$ consisting of the k training observations closest to x_0, usually measured using Euclidean distance (EQ 12):

$$d(x_i, x_0) = \sqrt{\sum_{j=1}^{p} (x_{ij} - x_{0j})^2}$$

The estimated probability that x_0 belongs to class j is (EQ 13):

$$P(Y = j \mid X = x_0) = \frac{1}{k} \sum_{i \in N_k(x_0)} I\,(y_i = j)$$

where I is an indicator function equal to 1 when the condition is true and 0 otherwise.

The predicted class is the one with the largest estimated probability (EQ 14):

$$\hat{y} = \underset{j}{\text{argmax}} P(Y = j \mid X = x_0)$$

The choice of k controls the bias–variance trade-off. Small k leads to low bias but high variance, while larger k is the opposite.

Cross Validation

Cross validation is a resampling technique used to evaluate the predictive performance of a machine learning model and estimate how well the model generalizes to unseen data. It is commonly used for model selection and hyperparameter tuning while reducing the risk of overfitting.

In k-fold cross validation, the dataset is randomly divided into k approximately equal subsets called folds. One fold is used as the validation set while the remaining $k - 1$ folds are used for training. This process is repeated k times so that each fold serves as the validation set exactly once.

For fold i, the model is trained on the training subset D_{-i} and evaluated on the validation subset D_i (EQ 15):

$$\text{Error}_i = L\left(y_i, \hat{f}_{-i}(x_i)\right)$$

where L is a loss function measuring prediction error and $\hat{f}_{-i}$ represents the model trained without fold i.

The overall cross validation error is computed as the average error across all folds (EQ 16):

$$CV_k = \frac{1}{k} \sum_{i=1}^{k} \text{Error}_i$$

A common choice is 5-fold or 10-fold cross validation because these values provide a good balance between computational efficiency and reliable performance estimation.

Cross validation is especially useful when selecting tuning parameters, such as the number of neighbors in KNN or the depth of a decision tree. The parameter combination with the best average validation performance is selected as the final model configuration.

Natural Cubic Splines

Natural cubic splines are a flexible regression technique used to model nonlinear relationships between predictor variables and a response variable. Instead of fitting a single straight line, the method fits multiple cubic polynomial functions joined together at specific points called knots.

For a predictor variable x with knots $\xi_1, \xi_2, \ldots, \xi_K$, a cubic spline is represented as a piecewise cubic polynomial (EQ 17):

$$f(x) = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \sum_{k=1}^{K} \gamma_k \, (x - \xi_k)_+^3$$

where

$$(x - \xi_k)_+^3 = \begin{cases} (x - \xi_k)^3, & x > \xi_k \\ 0, & x \leq \xi_k \end{cases}$$

Natural cubic splines add the additional constraint that the function becomes linear beyond the boundary knots. This reduces instability at the extremes of the predictor range and helps prevent overfitting.

The spline function remains smooth because the fitted curves are constrained to have continuous first and second derivatives at each knot.

Natural cubic splines are useful when the relationship between variables is not strictly linear but still requires a smooth and interpretable model.

Results

Preprocess Data

When processesing the data, I had to change many variables into factors. I also cleaned the names so that they would be easier to use when coding. Finally, I removed two columns, both relating to ID numbers, since those are unique to each row and have no impact on classification. This dataset has 13 variables that I will use and 7500 rows. The variable of interest is very unbalanced with about 93% of observations are legitimate and only 7% are fraudulent.

```
## Rows: 7500 Columns: 15
## ── Column specification
────────────────────────────────────────────────────────
## Delimiter: ","
## chr (6): transaction_id, user_id, transaction_type, payment_mode,
device_typ...
## dbl (9): transaction_amount, account_age_days, transaction_hour,
```

```
previous_fa...
##
## ℹ Use `spec()` to retrieve the full column specification for this data.
## ℹ Specify the column types or set `show_col_types = FALSE` to quiet this
message.

## [1] "Number of variables: 13"

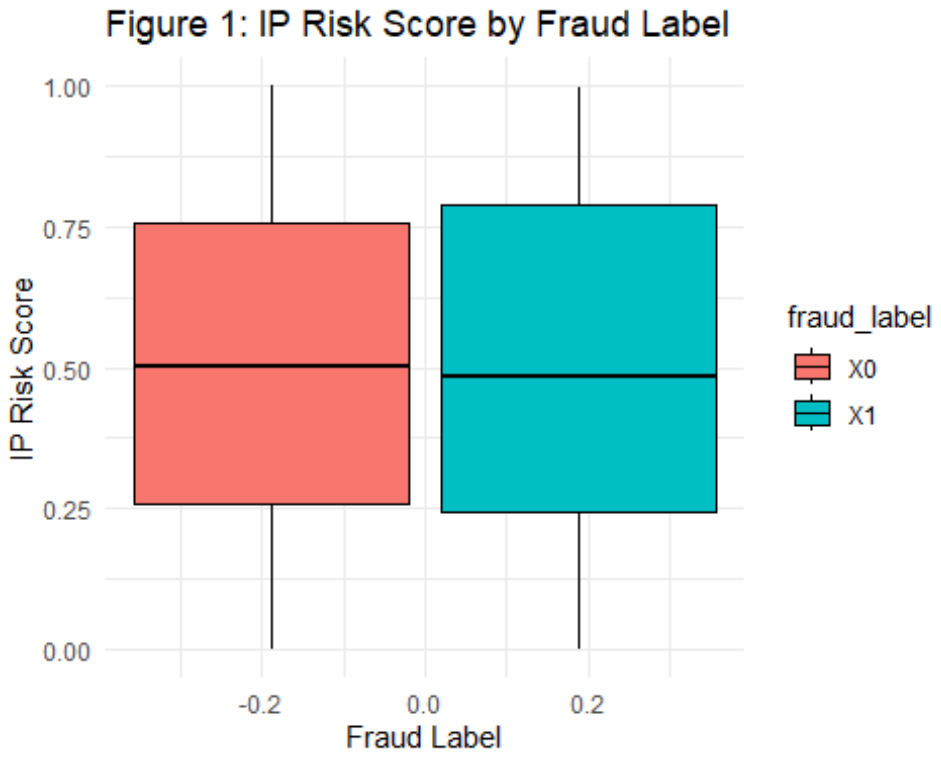
## [1] "Number of observations: 7500"

##
##     X0     X1
## 0.9348 0.0652

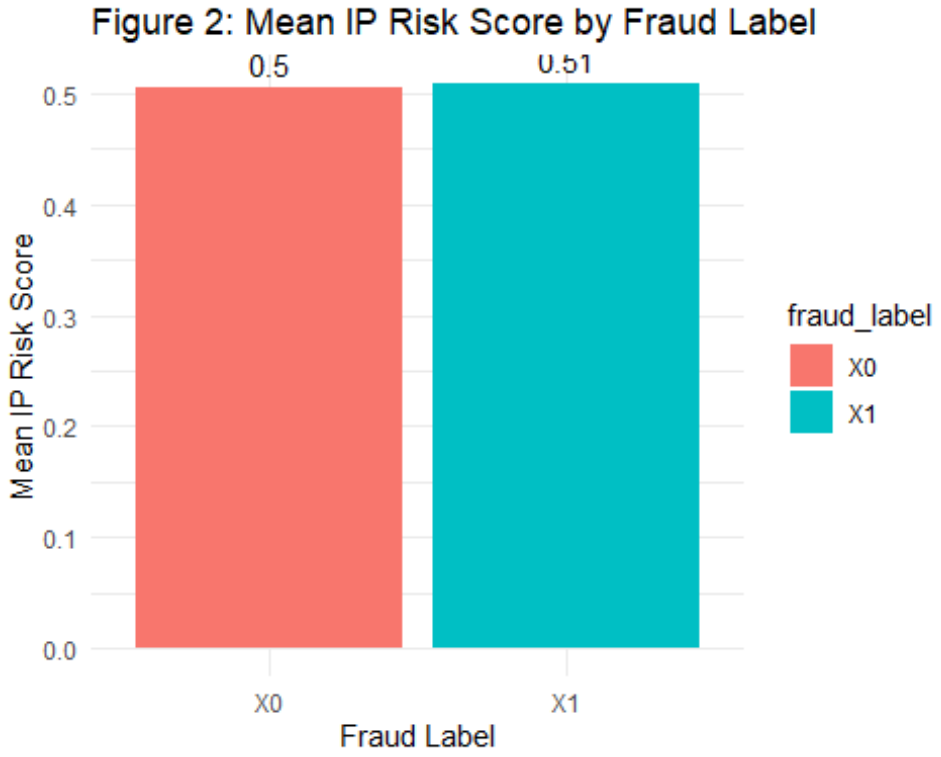
## Joining with `by = join_by(transaction_amount, transaction_type,
payment_mode,
## device_type, device_location, account_age_days, transaction_hour,
## previous_failed_attempts, avg_transaction_amount, is_international,
## ip_risk_score, login_attempts_last_24h, fraud_label)`
## Joining with `by = join_by(transaction_amount, transaction_type,
payment_mode,
## device_type, device_location, account_age_days, transaction_hour,
## previous_failed_attempts, avg_transaction_amount, is_international,
## ip_risk_score, login_attempts_last_24h, fraud_label)`
```

Exploratory Data Analysis

This boxplot shows the differences in IP risk score based on fradulent and legitimate transactions. The medians are very similar, however there is more varaince in the fraudulent transaction boxplot.



These boxplots show the mean of the ip risk score for each class. In the fraudulent class the mean is slightly higher than in the legitimate class.

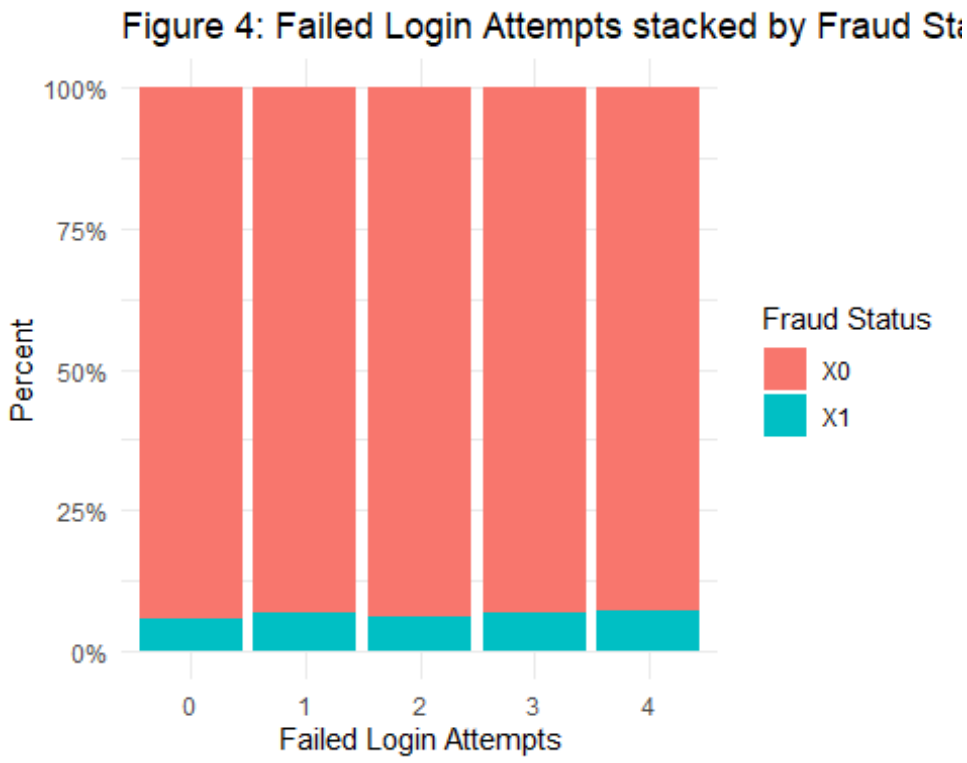


This violin plot shows the distribution and density of Average transaction amount in the two classes. Both plots are very similar showing that this varaible might not be useful in the classification models.

Figure 3: Violin Plot of Average Transaction Amount

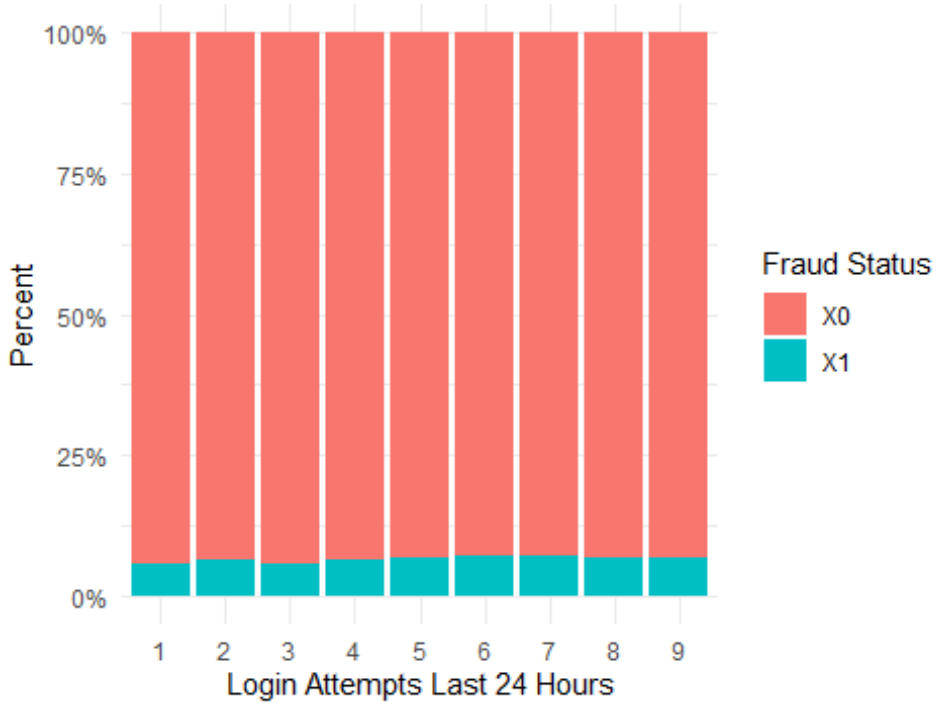


This bar chart shows the distribution of classes in the different values for failed login attempts. There is a slight trend where each addition failed login has more fraudulent transactions in it.

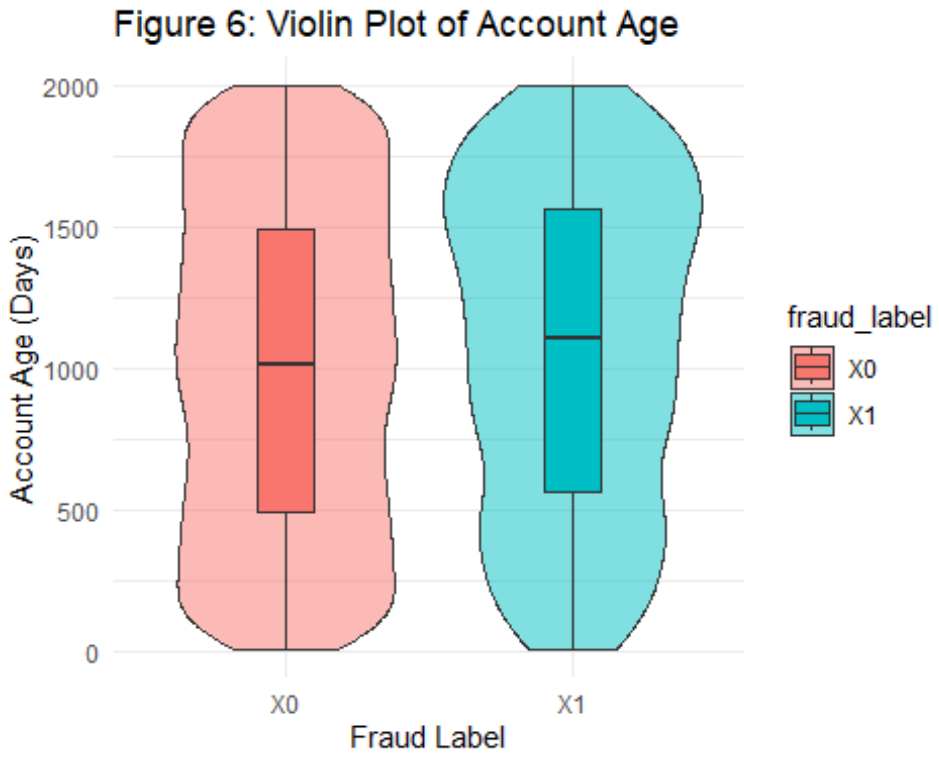


This bar chart shows the distribution of classes by each value of login attempts. There is a slight trend of more login attempts, the more fraudulent cases.

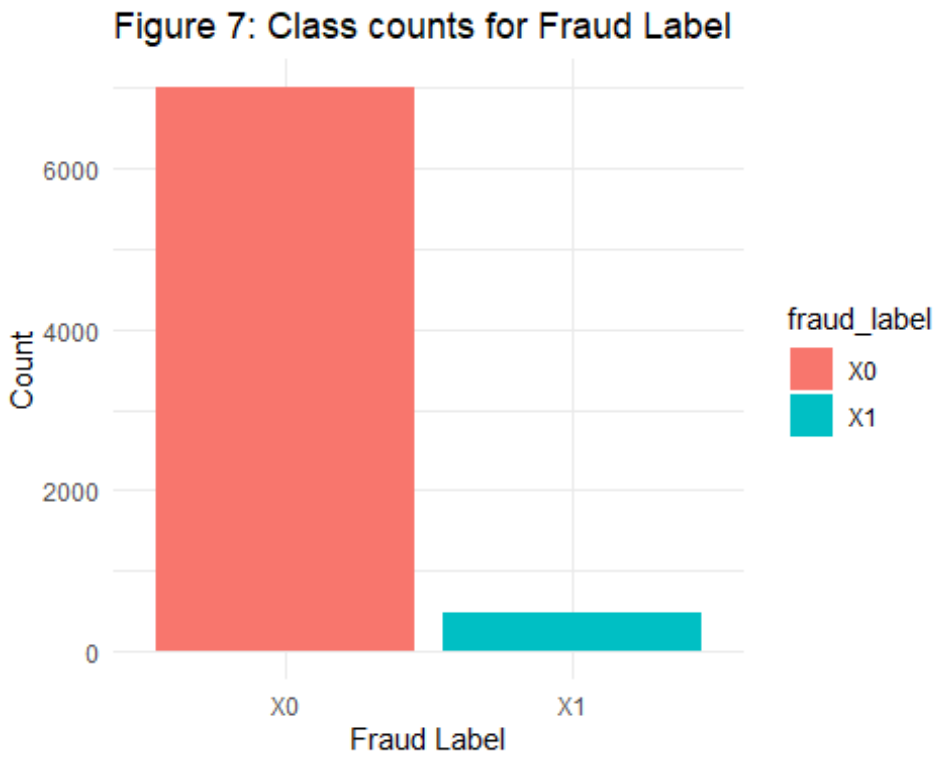
Figure 5: Login Attempts stacked by Fraud Status



This violin plot shows the distribution and density of Account Age between the two classes. There is a clear difference in the shapes of the two plots as well as the boxplots. This shows that account age might be a useful predictor variable in the models.



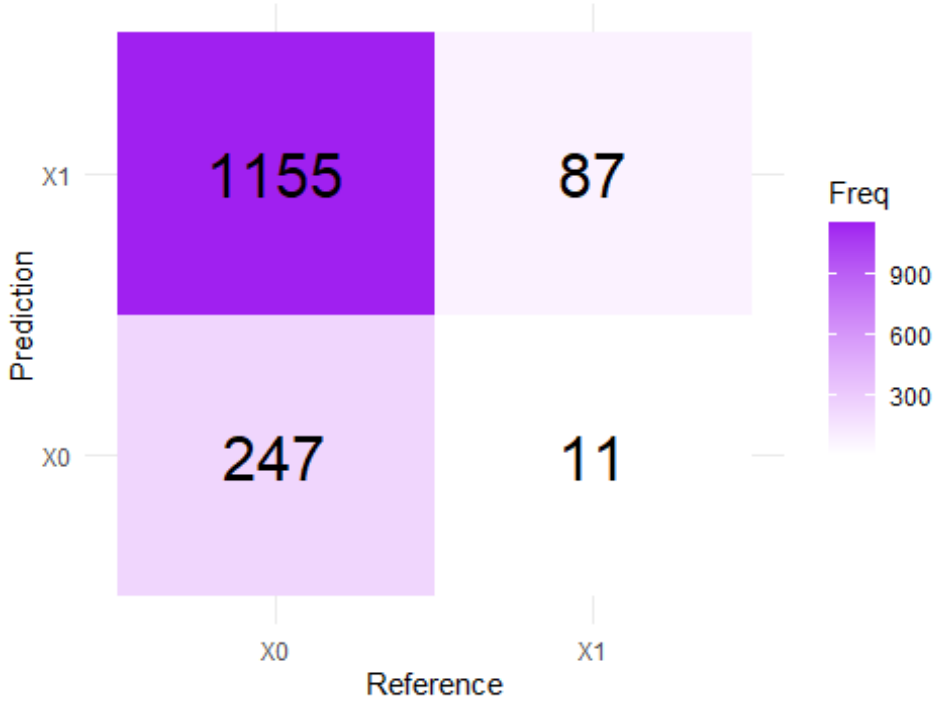
This bar chart shows the class imbalance in the dataset. X0 represents the legitimate class and it far outnumber the fraudulent class, X1. This shows that resampling techniques should be used to improve the model performance.



Logistic Regression

Logistic regression was selected as a base model because it is interpretable and well suited for binary classification problems like fraud detection. To improve model performance, I applied natural cubic splines to ip_risk_score and account_age_days so the model could capture nonlinear relationships in those variables. Since the dataset was imbalanced, I used class weights to place a greater emphasis on fraudulent transactions during training, helping reduce the bias toward the majority class. I also used an optimal classification threshold by using ROC curve analysis instead of the default 0.5 cutoff in order to improve the balance between sensitivity and specificity. The resulting model achieved strong fraud detection performance by identifying a large proportion of fraudulent transactions, although this came at the cost of an increased number of false positives, which is often acceptable in fraud detection applications where missing fraudulent activity can be more costly than incorrectly flagging legitimate transactions. In the confusion matrix X1 is the fraudulent class. It achieved a high sensitivity score of 89%, meaning that it was good at catching fraudulent cases.

Figure 8: Confusion Matrix Heatmap - Logistic Model



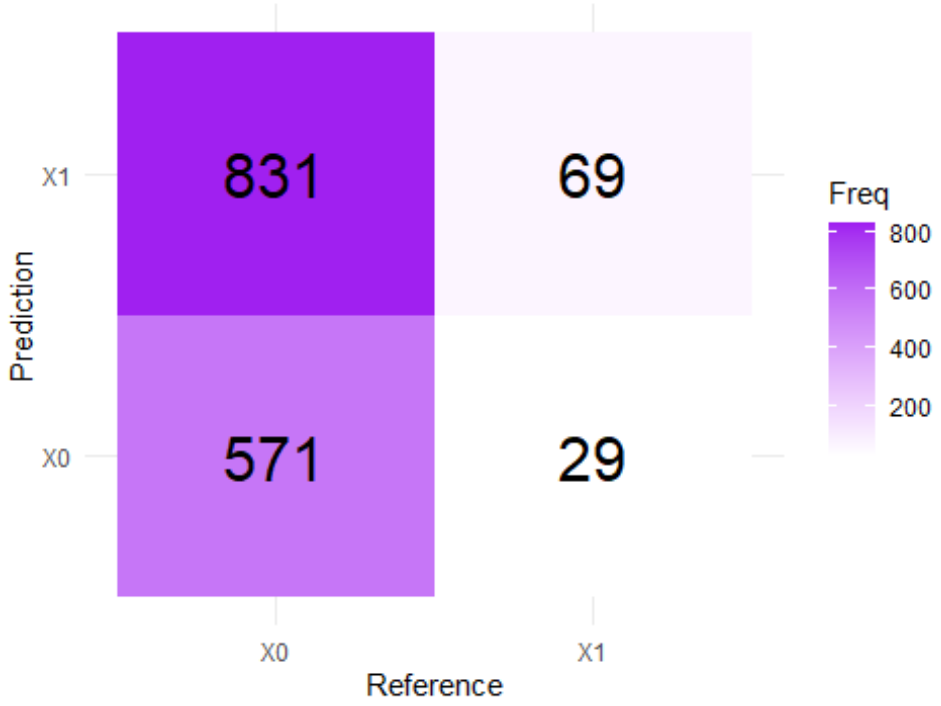
KNN

The k-nearest neighbors model was developed as a classification method that predicts fraud based on similarity between transactions in the predictor space. The model used ip_risk_score, previous_failed_attempts and account_age_days as predictors, with all variables centered and scaled to ensure equal influence during distance calculations. Because of the imbalance in teh dataset, downsampling was applied during the 10-fold cross validation to reduce bias toward legitimate transactions. Multiple tuning parameters, such as the maximum number of neighbors, distance metrics, and kernel functions were optimized using ROC performance to identify the best model configuration. The threshold was also optimized in this way. The final KNN model was able to detect fraudulent transactions reasonably well, although its performance was limited by the overlap between fraudulent and legitimate observations.

```
## Confusion Matrix and Statistics
##
##            Reference
## Prediction  X0  X1
##         X0 437  26
##         X1 685  52
##
##                Accuracy : 0.4075
##                  95% CI : (0.3795, 0.4359)
```

```
##     No Information Rate : 0.935
##     P-Value [Acc > NIR] : 1
##
##                   Kappa : 0.0114
##
##  Mcnemar's Test P-Value : <2e-16
##
##             Sensitivity : 0.66667
##             Specificity : 0.38948
##          Pos Pred Value : 0.07056
##          Neg Pred Value : 0.94384
##              Prevalence : 0.06500
##          Detection Rate : 0.04333
##    Detection Prevalence : 0.61417
##       Balanced Accuracy : 0.52807
##
##        'Positive' Class : X1
##
```

Figure 9: Confusion Matrix Heatmap - KNN Model



Random Forest

The random forest model is an ensemble learning method that combines many decision trees to improve classification accuracy. Stratified sampling was applied during training to deal with the class imbalance. The final model consisted of 1000 decision trees with an

mtry value of 3. Figure 10 shows the variable importance plot, which predictor varaibles were most influential when making decisions. The final threshold was decided using the highest F1-score. Despite its high accuracy, the model still faced challenges associated with class imbalance and predicted too many observations to be legitimate missing many fraud cases.

Figure 10: Variable Imp for Random Forest

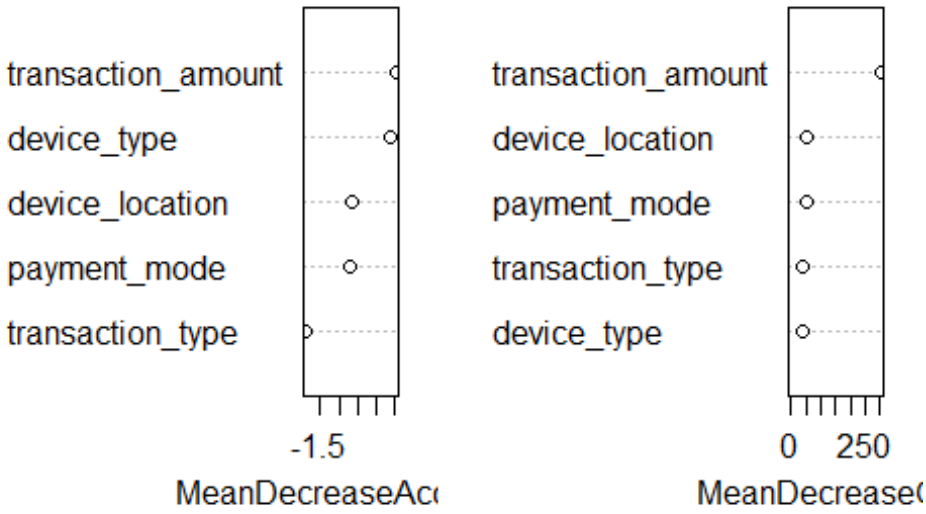
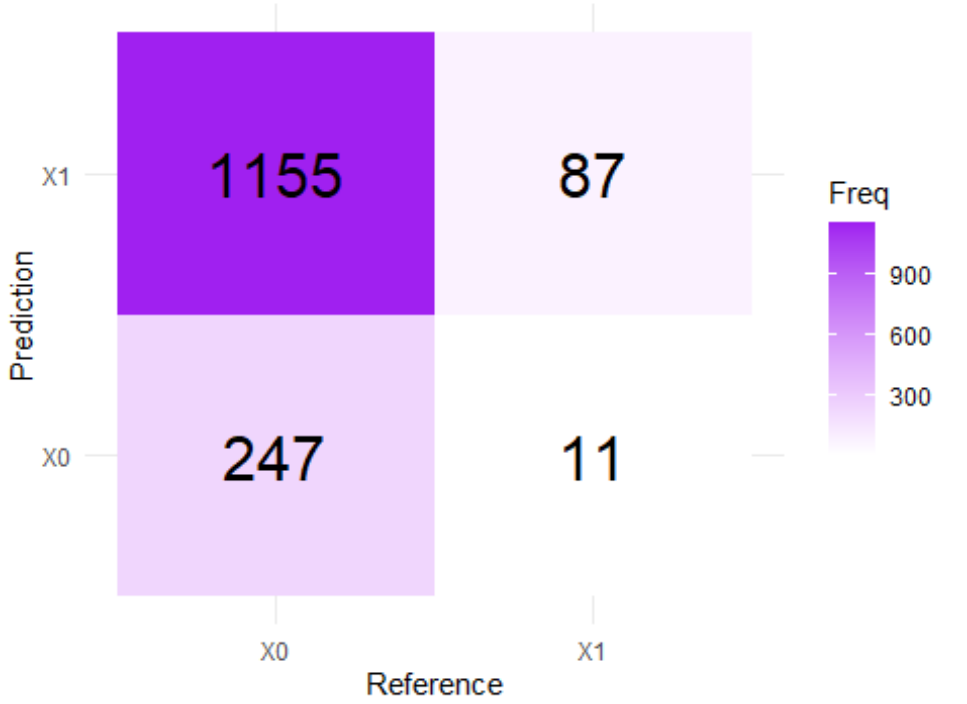


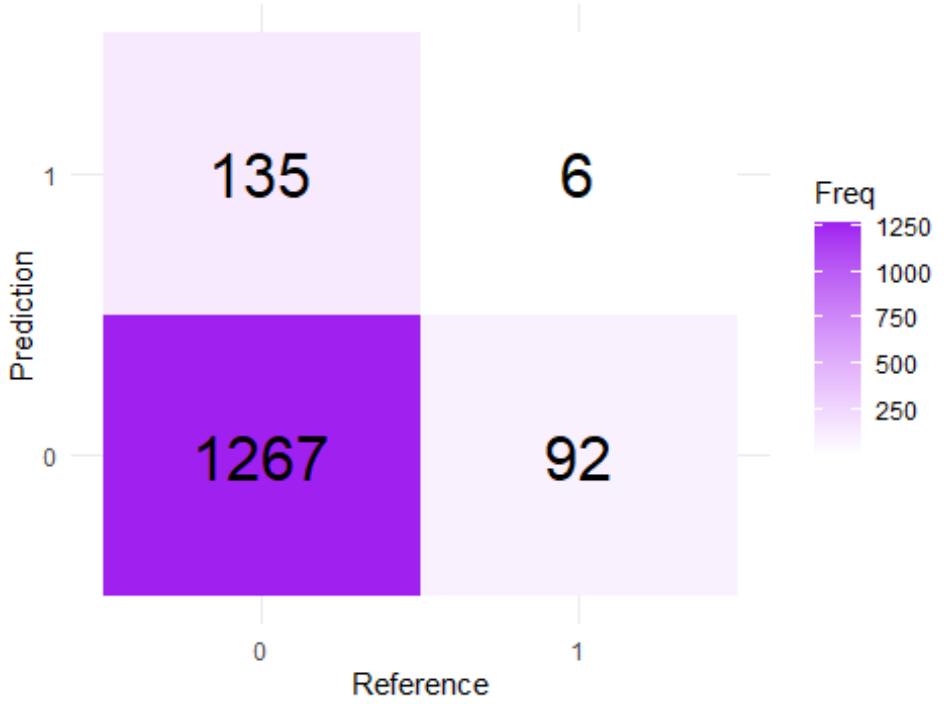
Figure 11: Confusion Matrix Heatmap - Random Forest



Boosted Tree

The boosted tree model was developed using gradient boosting, which builds many small decision trees sequentially so that each new tree attempts to improve on the errors made by the previous trees. The response variable was converted to a numeric binary format so the model could be fit using a Bernoulli distribution, making it appropriate for fraud classification. The model used 5,000 trees, a small shrinkage value of 0.001, and an interaction depth of 3. After generating predicted fraud probabilities on the test set, ROC curve analysis was used to select an optimal classification threshold instead of relying on the default 0.5 cutoff. The final boosted tree model was able to capture nonlinear relationships and interactions across the predictor variables. However, its performance was limited by the class imbalance in the dataset, as shown by its difficulty correctly identifying fraudulent transactions compared with legitimate ones.

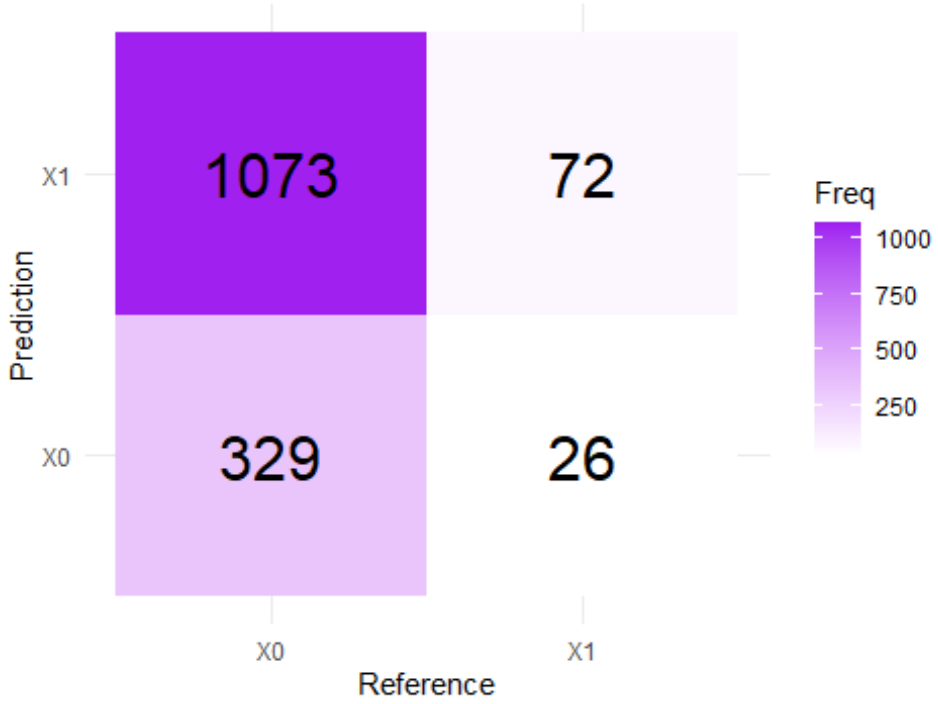
Figure 12: Confusion Matrix Heatmap - Boosted Tree M



BART

The Bayesian Additive Regression Trees model was developed as an ensemble learning method that combines many decision trees to model complex nonlinear relationships in the fraud detection dataset. Because the dataset was highly imbalanced, upsampling was applied to the training data so that fraudulent transactions were more equally represented during model training, helping the model better recognize patterns associated with fraud. The predictor variables were transformed into a model matrix format before training, and the final BART model was fit using 100 trees with posterior sampling to estimate fraud probabilities for each transaction. After generating predicted probabilities on the test set, a lower classification threshold of 0.05 was selected to improve the model's sensitivity toward fraudulent transactions. The BART model was effective at identifying nonlinear interactions and uncertainty within the data. However, despite its flexibility, the model still experienced challenges balancing fraud detection performance with false positive predictions due to the severe class imbalance present in the dataset.

Figure 13: Confusion Matrix Heatmap - BART Model



Model Comparisson

Figure 14 shows the Precision vs Sensitivity of each model on the test set.

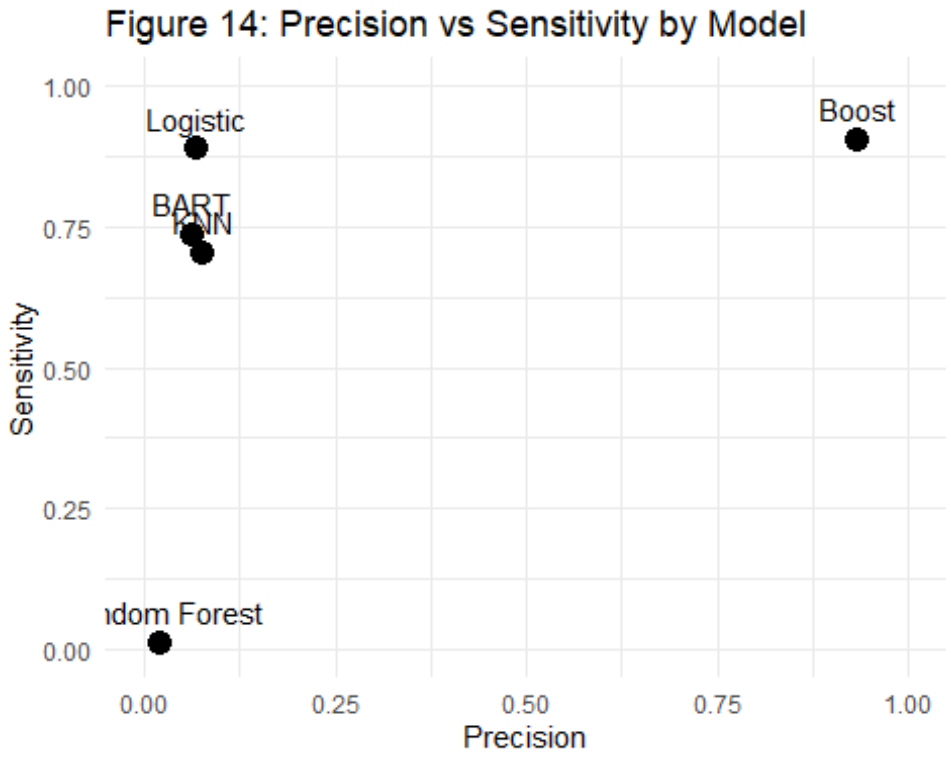
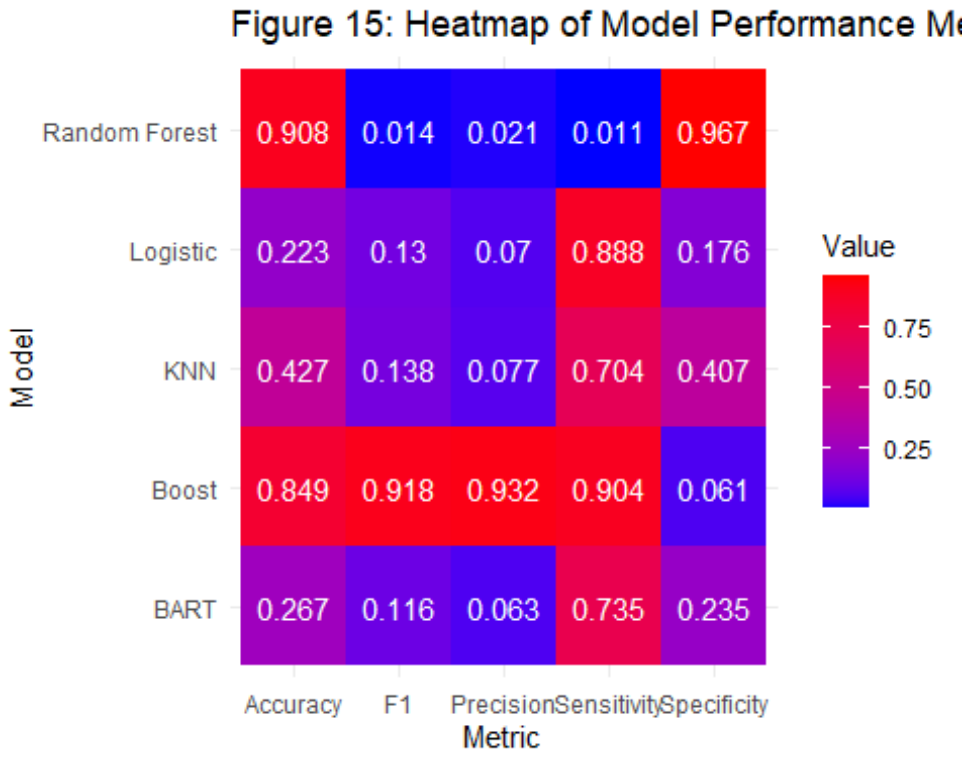


Figure 15 shows the heatmap of each model for each performance metric. Blue represents a low value and red represents a high value.



Discussion

Among all models evaluated, the boosted model performed best for fraud detection. Although it had lower overall accuracy, it achieved the highest sensitivity and F1 score, making it the most effective at identifying fraudulent transactions. In contrast, models such as Random Forest achieved high accuracy but failed to detect fraud, demonstrating that accuracy alone is not an appropriate metric for this problem. The boosted model looks like it performed the best however it has a very low specificity, meaning it predicted almost nothing as legitimate, therefore I still prefer the logistic model.

The most important variables were ip risk score, account age, and previous attempts. These variables make sense since a high ip risk score, a new account, and many failed attempts at buying are red flags for fraudulent behavior.

When building the models, there are many things that can go wrong. A major problem I faced was class imbalance. The dataset was very unbalanced having many more nonfraudulent transactions than fraudulent ones. There are resampling techniques that help solve this problem however they are not perfect. Another problem is overfitting. When your model does very well on the training set and then the performance drops significantly in testing. This is often because the model captured too much noise in training instead of finding general patterns in the data.

In the future, if I were to revisit this project, I would have liked to use a few more complex models. I believe that a deep model like a neural network or a more advance gradient boosting model would be successful in this type of problem. Another thing I'd like to do is make the dataset better. Models can only perform as well as the data allows, so I would try to improve the data through things like feature engineering, PCA, and feature selection.

Reference

Sources

James, Gareth, et al. An Introduction to Statistical Learning: with Applications in R. 2nd ed., Springer, 2023.

"Deciding Threshold for GLM Logistic Regression Model in R." GeeksforGeeks, 23 July 2025, https://www.geeksforgeeks.org/machine-learning/deciding-threshold-for-glm-logistic-regression-model-in-r/

"ns: Generate a Basis Matrix for Natural Cubic Splines." R Documentation, version 3.6.2, The R Foundation, https://www.rdocumentation.org/packages/splines/versions/3.6.2/topics/ns

"How to Calculate the OOB of Random Forest in R?" GeeksforGeeks, 23 July 2025, https://www.geeksforgeeks.org/r-machine-learning/how-to-calculate-the-oob-of-random-forest-in-r/

AI Usage

In this project I used ChatGPT to generate the lateX code in my methodology section. I also used it to help with some of the styling in my plots and tables.

Appendix

```{r read_data, warning=FALSE}
fraud_data <- read_csv("Digital_Payment_Fraud_Detection_Dataset.csv")
fraud_data <- clean_names(fraud_data)

fraud_data <- fraud_data |>
  mutate(
    fraud_label = as.factor(fraud_label),
    transaction_type = as.factor(transaction_type),
    payment_mode = as.factor(payment_mode),
    device_type = as.factor(device_type),
    is_international = as.factor(is_international),
    device_location = as.factor(device_location)
  )|>
  dplyr::select(-c(transaction_id,user_id))

levels(fraud_data$fraud_label) <- make.names(levels(fraud_data$fraud_label))

paste("Number of variables:",ncol(fraud_data))
paste("Number of observations:",nrow(fraud_data))
prop.table(table(fraud_data$fraud_label))
```

```{r split_data}
set.seed(2026)
d1 <- fraud_data|>
  group_by(fraud_label)|>
  sample_frac(0.8)

test <- anti_join(fraud_data,d1)
train <- d1|>
  group_by(fraud_label)|>
  sample_frac(0.8)
validation <- anti_join(d1,train)
```

```{r ip_risk_boxplot}
ggplot(fraud_data,
       aes(y=ip_risk_score, fill=fraud_label))+
  geom_boxplot(col="black")+
  labs(y="IP Risk Score",
       x= "Fraud Label",
       title= "Figure 1: IP Risk Score by Fraud Label")+
  theme_minimal()

```

These boxplots show the mean of the ip risk score for each class. In the fraudulent class the mean is slightly higher than in the legitimate class.

```{r mean_ip_risk}
fraud_data |>
  group_by(fraud_label)|>
  summarise(mean_ip_risk_score = mean(ip_risk_score))|>
  ggplot(aes(x = fraud_label, y = mean_ip_risk_score,fill = fraud_label)) +
  geom_col() +
  geom_text(
    aes(label = round(mean_ip_risk_score, 2)),
    vjust = -0.5
  ) +
  labs(
    x = "Fraud Label",
    y = "Mean IP Risk Score",
    title = "Figure 2: Mean IP Risk Score by Fraud Label"
  ) +
  theme_minimal()

```

```{r avg_trans_violin}

ggplot(fraud_data,
       aes(x = fraud_label, y = avg_transaction_amount,fill=fraud_label))+
  geom_violin(alpha = 0.5)+
  geom_boxplot(width = 0.2, outlier.shape = NA)+
  labs(
    y = "Average Transaction Amount",
    x = "Fraud Label",
    title = "Figure 3: Violin Plot of Average Transaction Amount"
  )+
  theme_minimal()
```

This bar chart shows the distribution of classes in the different values for failed login attempts. There is a slight trend where each addition failed login has more fraudulent transactions in it.

```{r login_atmpt_stacked_bar}
ggplot(fraud_data, aes(x = factor(previous_failed_attempts), fill = fraud_label)) +
  geom_bar(position = "fill") +
  scale_y_continuous(labels = scales::percent) +
  labs(x = "Failed Login Attempts",
       y = "Percent",
       fill = "Fraud Status",
       title = "Figure 4: Failed Login Attempts stacked by Fraud Status") +
  theme_minimal()
```

This bar chart shows the distribution of classes by each value of login attempts. There is a slight trend of more login attempts, the more fraudulent cases.

```{r login_attempts}
ggplot(fraud_data, aes(x = factor(login_attempts_last_24h), fill = fraud_label)) +
  geom_bar(position = "fill") +
  scale_y_continuous(labels = scales::percent) +
  labs(x = "Login Attempts Last 24 Hours",
       y = "Percent",
       fill = "Fraud Status",
       title = "Figure 5: Login Attempts stacked by Fraud Status") +
  theme_minimal()
```

```{r account_age}

ggplot(fraud_data,
       aes(x = fraud_label, y = account_age_days,fill=fraud_label))+
  geom_violin(alpha = 0.5)+
  geom_boxplot(width = 0.2, outlier.shape = NA)+
  labs(
    y = "Account Age (Days)",
    x = "Fraud Label",
    title = "Figure 6: Violin Plot of Account Age"
  )+
  theme_minimal()
```

This bar chart shows the class imbalance in the dataset. X0 represents the legitimate class and it far outnumber the fraudulent class, X1. This shows that resampling techniques should be used to improve the model performance.

```{r figure_7}
ggplot(fraud_data,
       aes(x = fraud_label, fill = fraud_label))+
  geom_bar()+
  labs(
    x = "Fraud Label",
    y = "Count",
    title = "Figure 7: Class counts for Fraud Label"
  )+
  theme_minimal()
```

```{r logistic_model3}
library(splines)

logistic3 <- glm(fraud_label ~
                   ns(ip_risk_score, 3) +
                   ns(account_age_days, 3) +
                   previous_failed_attempts,
                 data = train,
                 family = "binomial",
                 weights = ifelse(train$fraud_label == "X1", 5, 1))

Lprobs3 <- predict(logistic3,validation,type="response")

# find optimal threshold

# roc_obj <- roc(validation$fraud_label, Lprobs3)
# plot(roc_obj)
#
# coords(roc_obj, "best", ret = "threshold")

Pred3 <- ifelse(Lprobs3 >= 0.2623522, "X1", "X0")

Pred3 <- factor(Pred3, levels = c("X0", "X1"))

Actual3 <- factor(validation$fraud_label, levels = c("X0", "X1"))

#confusionMatrix(Pred3, Actual3, positive = "X1")

```

```{r best_logistic}
LprobsB <- predict(logistic3,test,type="response")

# find optimal threshold

# roc_obj <- roc(test$fraud_label, LprobsB)
# plot(roc_obj)
#
# coords(roc_obj, "best", ret = "threshold")

PredB <- ifelse(LprobsB >= 0.2280686, "X1", "X0")

PredB <- factor(PredB, levels = c("X0", "X1"))

ActualB <- factor(test$fraud_label, levels = c("X0", "X1"))

cm_log <- confusionMatrix(PredB, ActualB, positive = "X1")

cm <- cm_log$table
cm_df <- as.data.frame(cm)

ggplot(cm_df, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile() +
  geom_text(aes(label = Freq), size = 8, color="black") +
  scale_fill_gradient(low = "white", high = "purple") +
  theme_minimal() +
  labs(
    title = "Figure 8: Confusion Matrix Heatmap - Logistic Model"
  )

```

```{r knn3}

# cross-validation setup
ctrl_kfold3 <- caret::trainControl(
  method = "cv",
  number = 10,
  classProbs = TRUE,
  summaryFunction = caret::twoClassSummary,
  sampling = "down"
)

# tuning grid
kknn_grid <- expand.grid(
  kmax = seq(5, 25, by = 2),
  distance = c(1, 2),
  kernel = c("rectangular", "triangular", "optimal")
)

# final KNN model
knn_model3 <- caret::train(
  fraud_label ~ ip_risk_score + previous_failed_attempts + account_age_days,
  data = train,
  method = "kknn",
  tuneGrid = kknn_grid,
  trControl = ctrl_kfold3,
  preProcess = c("center", "scale"),
  metric = "ROC"
)

#print(knn_model3)
#plot(knn_model3)
#knn_model3$bestTune

# predicted probabilities for fraud
knn_probs3 <- predict(knn_model3, newdata = validation, type = "prob")[, "X1"]
actual <- factor(validation$fraud_label, levels = c("X0", "X1"))

# threshold search
results <- data.frame()

for (t in seq(0.05, 0.40, by = 0.01)) {

  pred <- factor(ifelse(knn_probs3 >= t, "X1", "X0"),
                 levels = c("X0", "X1"))

  cm <- caret::confusionMatrix(pred, actual, positive = "X1")

  results <- rbind(results, data.frame(
  results <- rbind(results, data.frame(
    threshold = t,
    sensitivity = cm$byClass["Sensitivity"],
    specificity = cm$byClass["Specificity"],
    precision = cm$byClass["Pos Pred Value"],
    F1 = cm$byClass["F1"],
    accuracy = cm$overall["Accuracy"]
  ))
}

results_filtered <- results[results$precision >= 0.10, ]

if (nrow(results_filtered) == 0) {
  best_row <- results[which.max(results$F1), ]
} else {
  best_row <- results_filtered[which.max(results_filtered$F1), ]
}

best_t <- best_row$threshold
# best_t


final_pred3 <- factor(ifelse(knn_probs3 >= best_t, "X1", "X0"),
                      levels = c("X0", "X1"))

caret::confusionMatrix(final_pred3, actual, positive = "X1")
```

```{r knn_test}


# get predicted probabilities for fraud (X1)
knn_probs_test <- predict(knn_model3, newdata = test, type = "prob")[, "X1"]

# actual values
actual_test <- factor(test$fraud_label, levels = c("X0", "X1"))

# apply chosen threshold (replace best_t if needed)
best_t <- best_t  # or manually: best_t <- 0.30 (example)

# predicted classes
knn_pred_test <- ifelse(knn_probs_test >= best_t, "X1", "X0")
knn_pred_test <- factor(knn_pred_test, levels = c("X0", "X1"))

# confusion matrix
cm_knn <- caret::confusionMatrix(knn_pred_test, actual_test, positive = "X1")



cm <- cm_knn$table
cm_df <- as.data.frame(cm)

ggplot(cm_df, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile() +
  geom_text(aes(label = Freq), size = 8, color="black") +
  scale_fill_gradient(low = "white", high = "purple") +
  theme_minimal() +
  labs(
    title = "Figure 9: Confusion Matrix Heatmap - KNN Model"
  )
```

```{r data_for_trees}
set.seed(20)

split_data <- initial_split(fraud_data, prop = 0.8, strata = fraud_label)

train_tree_no_cv <- training(split_data)
test_tree <- testing(split_data)

split2 <- initial_split(train_tree_no_cv, prop = 0.8, strata = fraud_label)
train_tree <- training(split2)
validation_tree <- testing(split2)

```

```{r randomForest4}
library(randomForest)
library(caret)

set.seed(2026)


# class counts
class_counts <- table(train_tree_no_cv$fraud_label)
minority_n <- min(class_counts)

# less aggressive balanced sampling than model 3
rf_model4 <- randomForest(
  fraud_label ~ transaction_amount + transaction_type + payment_mode +
    device_type + device_location,
  data = train_tree_no_cv,
  ntree = 1000,
  mtry = 3,
  importance = TRUE,
  strata = train_tree_no_cv$fraud_label,
  sampsize = c(minority_n * 2, minority_n),
  nodesize = 1
)

# OOB error rate
# oob_error_rate4 <- rf_model4$err.rate[nrow(rf_model4$err.rate), "OOB"]
# cat("OOB Error Rate:", round(oob_error_rate4 * 100, 2), "%\n")
#
# # OOB confusion matrix from model
# confusion_matrix4 <- rf_model4$confusion
# print(confusion_matrix4)

# variable importance
#print(importance(rf_model4))
varImpPlot(rf_model4, main= "Figure 10: Variable Imp for Random Forest")

# OOB error plot
# plot(rf_model4)



rf_probs4 <- predict(rf_model4, newdata = train_tree_no_cv, type = "prob")[, "X1"]

actual4 <- factor(train_tree_no_cv$fraud_label, levels = c("X0", "X1"))

threshold_results4 <- data.frame()

for (t in seq(0.10, 0.50, by = 0.05)) {

  pred_t <- factor(ifelse(rf_probs4 >= t, "X1", "X0"),
                   levels = c("X0", "X1"))

  cm_t <- caret::confusionMatrix(pred_t, actual4, positive = "X1")

  threshold_results4 <- rbind(
    threshold_results4,
    data.frame(
      threshold = t,
      sensitivity = cm_t$byClass["Sensitivity"],
      specificity = cm_t$byClass["Specificity"],
      precision   = cm_t$byClass["Pos Pred Value"],
      F1          = cm_t$byClass["F1"],
      accuracy    = cm_t$overall["Accuracy"]
    )
  )
}

# threshold_results4
# threshold_results4[order(-threshold_results4$F1), ]


threshold_results4_fine <- data.frame()

for (t in seq(0.05, 0.90, by = 0.01)) {

  pred_t <- factor(ifelse(rf_probs4 >= t, "X1", "X0"),
                   levels = c("X0", "X1"))

  cm_t <- caret::confusionMatrix(pred_t, actual4, positive = "X1")
```

```
  threshold_results4_fine <- rbind(
    threshold_results4_fine,
    data.frame(
      threshold = t,
      sensitivity = cm_t$byClass["Sensitivity"],
      specificity = cm_t$byClass["Specificity"],
      precision   = cm_t$byClass["Pos Pred Value"],
      F1          = cm_t$byClass["F1"],
      accuracy    = cm_t$overall["Accuracy"]
    )
  )
}

threshold_results4_fine[order(-threshold_results4_fine$F1), ]


best_t4 <- threshold_results4_fine$threshold[which.max(threshold_results4_fine$F1)]
best_t4

final_pred4 <- factor(ifelse(rf_probs4 >= best_t4, "X1", "X0"),
                      levels = c("X0", "X1"))

cm_rf4 <- caret::confusionMatrix(final_pred4, actual4, positive = "X1")
#cm_rf4
```

```{r testRandomForest}
rf_probs_test <- predict(rf_model4, newdata = test_tree, type = "prob")[, "X1"]

actual_test <- factor(test_tree$fraud_label, levels = c("X0", "X1"))

final_pred_test <- factor(ifelse(rf_probs_test >= 0.59, "X1", "X0"),
                          levels = c("X0", "X1"))

cm_rf <- caret::confusionMatrix(final_pred_test, actual_test, positive = "X1")

cm <- cm_log$table
cm_df <- as.data.frame(cm)

ggplot(cm_df, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile() +
  geom_text(aes(label = Freq), size = 8, color="black") +
  scale_fill_gradient(low = "white", high = "purple") +
  theme_minimal() +
  labs(
    title = "Figure 11: Confusion Matrix Heatmap - Random Forest Model"
  )
```

```{r boosting, warning=FALSE}
library(gbm)

# change y var to make numeric
boost_train <- d1|>
  mutate(fraud_label = ifelse(fraud_label == "X0", 0,1))
boost_test <- test|>
  mutate(fraud_label = ifelse(fraud_label == "X0", 0,1))

boost_model <- gbm(fraud_label ~ ., data = boost_train, distribution = "bernoulli", n.trees = 5000, interaction.depth = 3, shrinkage= 0.001)

yhat_boost <- predict(boost_model, newdata = boost_test, n.trees = 5000, type = "response")

actual <- factor(boost_test$fraud_label)

roc_obj <- roc(actual, yhat_boost, levels = c(0, 1))

best_thresh <- as.numeric(coords(roc_obj, "best", ret = "threshold"))
best_thresh

yhat_class <- ifelse(yhat_boost > best_thresh, 1, 0)

cm_boost <- confusionMatrix(as.factor(yhat_class),as.factor(boost_test$fraud_label))

cm <- cm_boost$table
cm_df <- as.data.frame(cm)

ggplot(cm_df, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile() +
  geom_text(aes(label = Freq), size = 8, color="black") +
  scale_fill_gradient(low = "white", high = "purple") +
  theme_minimal() +
  labs(
    title = "Figure 12: Confusion Matrix Heatmap - Boosted Tree Model"
  )
```

```{r bart2}
bart_train <- d1 |>
  mutate(fraud_label = ifelse(fraud_label == "X0", 0, 1)) |>
  as.data.frame()

bart_test <- test |>
  mutate(fraud_label = ifelse(fraud_label == "X0", 0, 1)) |>
  as.data.frame()

train_up <- upSample(
  x = bart_train |> dplyr::select(-fraud_label),
  y = factor(bart_train$fraud_label, levels = c(0, 1))
)

# upSample renames response to "Class"
train_up <- train_up |>
  mutate(fraud_label = ifelse(Class == "1", 1, 0)) |>
  dplyr::select(-Class)


xtrain2 <- model.matrix(fraud_label ~ . - 1, data = train_up)
ytrain2 <- train_up$fraud_label

xtest2 <- model.matrix(fraud_label ~ . - 1, data = bart_test)
ytest2 <- bart_test$fraud_label


set.seed(2026)

bart_model2 <- pbart(
  x.train = xtrain2,
  y.train = ytrain2,
  x.test  = xtest2,
  ntree = 100,
  ndpost = 1000,
  nskip = 200
)

bart_prob2 <- bart_model2$prob.test.mean

actual2 <- factor(ifelse(ytest2 == 1, "X1", "X0"), levels = c("X0", "X1"))


bart_pred2 <- ifelse(bart_prob2 >= 0.05, "X1", "X0")
bart_pred2 <- factor(bart_pred2, levels = c("X0", "X1"))

cm_bart <- confusionMatrix(bart_pred2, actual2, positive = "X1")
```

```
bart_prob2 <- bart_model2$prob.test.mean

actual2 <- factor(ifelse(ytest2 == 1, "X1", "X0"), levels = c("X0", "X1"))


bart_pred2 <- ifelse(bart_prob2 >= 0.05, "X1", "X0")
bart_pred2 <- factor(bart_pred2, levels = c("X0", "X1"))

cm_bart <- confusionMatrix(bart_pred2, actual2, positive = "X1")

cm <- cm_bart$table
cm_df <- as.data.frame(cm)

ggplot(cm_df, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile() +
  geom_text(aes(label = Freq), size = 8, color="black") +
  scale_fill_gradient(low = "white", high = "purple") +
  theme_minimal() +
  labs(
    title = "Figure 13: Confusion Matrix Heatmap - BART Model"
  )
```

Model Comparisson

```{r result_of_test}
get_metrics <- function(cm) {
  data.frame(
    Accuracy = as.numeric(cm$overall["Accuracy"]),
    Sensitivity = as.numeric(cm$byClass["Sensitivity"]),
    Specificity = as.numeric(cm$byClass["Specificity"]),
    Precision = as.numeric(cm$byClass["Pos Pred Value"]),
    F1 = as.numeric(cm$byClass["F1"])
  )
}

model_results <- rbind(
  cbind(Model = "Logistic", get_metrics(cm_log)),
  cbind(Model = "KNN", get_metrics(cm_knn)),
  cbind(Model = "Random Forest", get_metrics(cm_rf)),
  cbind(Model = "BART", get_metrics(cm_bart)),
  cbind(Model = "Boost", get_metrics(cm_boost))
)

rownames(model_results) <- NULL
```

```{r scatter_results}
ggplot(model_results, aes(x = Precision, y = Sensitivity, label = Model)) +
  geom_point(size = 4) +
  geom_text(vjust = -0.8) +
  theme_minimal() +
  labs(
    title = "Figure 14: Precision vs Sensitivity by Model",
    x = "Precision",
    y = "Sensitivity"
  ) +
  xlim(0, 1) +
  ylim(0, 1)
```

Figure 15 shows the heatmap of each model for each performance metric. Blue represents a low value and red represents a high value.

```{r result_heatmap}


model_long <- model_results |>
  pivot_longer(
    cols = c(Accuracy, Sensitivity, Specificity, Precision, F1),
    names_to = "Metric",
    values_to = "Value"
  )


ggplot(model_long, aes(x = Metric, y = Model, fill = Value)) +
  geom_tile() +
  geom_text(aes(label = round(Value, 3)), color = "white", size = 4) +
  scale_fill_gradient(low = "blue", high = "red") +
  theme_minimal() +
  labs(
    title = "Figure 15: Heatmap of Model Performance Metrics",
    x = "Metric",
    y = "Model"
  )
```